Statistical Estimation of Canteen Arrival Overload Probability Using Poisson Distribution

Estimating and predicting lunch-time overcrowding in a university canteen: Poisson models, Bayesian updating and forecast validation

Faraz Akhtar Khan (26MCA0197), Team in-charge Mayur Krishna (26MCA0191)
Malhar Bhosale (26MCA0199)

PAMAT501 Probability and Statistics (B2+TB2+TBB2): Course-Based Project · Faculty: Dr. Manimaran A · 26 October 2026

Contents

Abstract **2**

Keywords **2**

1 Introduction **2**
1.1 Aim . . . 2
1.2 Objectives . . . 3

2 Preliminaries **3**

3 Literature Review **4**

4 Data and Data Reference **5**

5 Main Content: Analysis and Results **6**
5.1 Arrival rate per slot . . . 6
5.2 Does the Poisson model fit? . . . 7
5.3 Overload probability and risk . . . 7
5.4 Least-crowded time . . . 8
5.5 Day-specific forecast by regression . . . 8
5.6 Peak-time probability . . . 9
5.7 Same-day Bayesian update . . . 9
5.8 Validation on unseen days . . . 10

6 Comparison **11**
6.1 Poisson versus Poisson–Gamma . . . 11
6.2 Forecast model versus the per-slot average . . . 11
6.3 This work versus existing studies . . . 12

7 Discussion **13**

8 Conclusion **13**

References **13**

Appendix: R Code **15**

Abstract

University canteens see sharp, short-lived rises in arrivals during the lunch peak. This project asks a practical question: *what is the probability that student arrivals exceed what the serving counters can handle in a particular 15-minute slot, and when will today's peak occur?* Arrivals in each slot are modelled as Poisson counts; a separate rate λ_t is estimated for every slot with an exact confidence interval, the Poisson assumption is tested slot by slot, and the overload probability $P(X_t > C)$ is converted into low, medium and high risk labels. The analysis is then extended to prediction: multiple linear regression gives day-specific rates for each weekday and for exam weeks, a Poisson–Gamma model allows for day-to-day variation, simulation gives the probability that each slot is the day's peak, and Bayes' theorem updates the forecast from the first counts of the day. Every forecast is checked on days the model has not seen. On a reproducible synthetic semester of 90 working days, the riskiest slot is 13:15-13:30 (P = 73.9% at C = 30); on a regular Monday the peak falls in 13:00-13:15 or 13:15-13:30 with probability 75%; forecast intervals cover 93.7% of unseen counts and the busiest slot is named correctly 40% of the time against 13% by chance.

Keywords

Poisson distribution; arrival rate estimation; overload probability; chi-square goodness of fit; multiple linear regression; negative binomial; Bayes' theorem; hypothesis testing; peak-time forecasting; university canteen.

1 Introduction

University canteens experience changing arrival intensity during lunch. When more students arrive in a short interval than the counters can serve, queues form, the dining area becomes crowded and popular dishes run out. Decisions about when to eat, how many counters to open and when to stagger breaks are usually taken by intuition rather than from measured arrival behaviour.

Earlier canteen studies model the whole system: pedestrian movement, window choice and seating [21, 22] or queueing models that assume Poisson arrivals [5, 11, 25]. These models leave a simple question unanswered: how likely is a given slot to exceed capacity, and when is the peak? This project answers it with the probability and statistics of the PAMAT501 course. It focuses on **arrival overload risk, not exact waiting time**.

1.1 Aim

To estimate, for each short time slot of the lunch period, the probability that student arrivals at a university canteen exceed the serving capacity, and to predict when the day's peak will occur, using the Poisson distribution and statistical tests that are checked against the data rather than assumed.

1.2 Objectives

1. **Collect and organise arrival data:** arrival counts in fixed 15-minute slots across m working days, validated and cleaned.
2. **Estimate time-specific rates:** $\hat{\lambda}_t$ for every slot with an exact confidence interval, and test whether the Poisson model fits.
3. **Calculate the overload probability:** $P(X_t > C)$ for a chosen capacity C, with its uncertainty, and label each slot low, medium or high risk.
4. **Identify useful time windows:** the least-crowded slot, with a test that it is genuinely quieter, and the capacity needed at the peak.
5. **Predict the peak time:** day-specific overload probabilities, the probability that each slot is the peak, and an update of the forecast from the day's first counts.
6. **Validate the forecasts:** forecast each day from earlier days only and test the forecasts with hypothesis tests.

2 Preliminaries

This section collects the probability and statistics used in the project.

Poisson distribution. If students arrive independently at a constant rate during a short slot, the number of arrivals X_t follows a Poisson distribution:

$$P(X_t = x) = \frac{e^{-\lambda_t}\lambda_t^x}{x!}, \qquad x = 0, 1, 2, \ldots, \qquad E[X_t] = \mathrm{Var}(X_t) = \lambda_t.$$

Estimation. For slot t observed on m days, the maximum-likelihood estimate is the sample mean, $\hat{\lambda}_t = \sum x_i/m$. Since $S = \sum x_i \sim \text{Poisson}(m\lambda_t)$, the exact (Garwood) confidence interval is $\left[\chi^2_{\alpha/2}(2S)/2m,\ \chi^2_{1-\alpha/2}(2S+2)/2m\right]$ [8].

Overload probability. With capacity C (threshold $K = C + 1$),

$$P(X_t > C) = P(X_t \geq K) = 1 - \sum_{x=0}^{C} \frac{e^{-\hat{\lambda}_t}\hat{\lambda}_t^x}{x!}.$$

A slot is labelled *Low* if this is below 20%, *High* above 50% and *Medium* otherwise (project decision rules).

Gamma, negative binomial and Bayes' theorem. If a whole day is busier or quieter by a factor $W \sim \text{Gamma}(1/v, 1/v)$ (mean 1, variance v), the theorem of total probability gives $P(X_t = x) = \int_0^\infty \text{Poisson}(x; w\lambda_t)\, g(w)\, dw$, a negative binomial distribution with mean λ_t and variance $\lambda_t + v\lambda_t^2$ [10]. After observing counts $x_1, \ldots, x_k$ with forecasts $\lambda_1, \ldots, \lambda_k$, Bayes' theorem gives the posterior $W \mid x \sim \text{Gamma}(a + \sum x_t,\ a + \sum \lambda_t)$ with $a = 1/v$.

Hypothesis tests. Table 1 lists the tests [12] and their syllabus modules.

Table 1: Statistical tests used in the project

Test	Used for	Module
Chi-square goodness of fit (categories merged to $E \geq 5$ [6])	Do counts follow Poisson($\hat{\lambda}$)?	M5
Index of dispersion $D = (m-1)s^2/\bar{x} \sim \chi^2_{m-1}$	Is variance equal to the mean?	M5
Exact conditional binomial test [17]	Is the quietest slot really quieter?	M2, M5
t-tests on regression coefficients	Do slot, weekday and exam effects matter?	M3, M5
One-way ANOVA, Kruskal–Wallis	Do weekdays differ in daily arrivals?	M5
Welch t-test, Mann–Whitney U	Are exam days quieter?	M5
z-test of a single proportion	Interval coverage; peak-slot hit rate	M4
Paired t-test, sign test	Is the forecast more accurate than the per-slot average?	M5
Kolmogorov–Smirnov, chi-square	Are the forecast distributions calibrated?	M5
Pearson and Spearman correlation	Predicted vs actual; carry-over between slots	M3
Wald–Wolfowitz run test	Do busy days follow busy days?	M5

3 Literature Review

Tang et al. [21] counted arrivals in 5-minute segments at a Beihang University canteen and built a pedestrian-flow model of the whole meal process; they note that arrivals are Poisson within short periods with a rate that changes every few minutes, but the arrival rate is an input to their simulation, not a quantity they test or predict. Queueing studies set serving windows from assumed Poisson arrivals [5, 25]; Jiao et al. [11] simulate one constant rate for the whole lunch. Brown et al. [3] and Kim and Whitt [13] show that Poisson arrivals must be tested, and that long intervals and pooled days cause overdispersion; Green et al. [9] and Leemis [14] justify short, piecewise-constant rates; Cameron and Trivedi [4] give overdispersion tests. Forecasting studies predict arrival rates for call centres and ambulance services and update them during the day [1, 15, 20, 24]; Ibrahim et al. [10] survey them. Shah et al. [19] forecast campus dining totals per day, and Variskhanov and Arreeras [23] issue a day-level crowd badge from Wi-Fi data. The Brier score [2] and a skill score against a reference forecast [16] are standard checks for probability forecasts.

Research gaps. (1) No per-slot overload probability for a canteen; (2) the Poisson assumption is built in rather than tested; (3) no prediction of a specific day's rates or peak time, with or without the day's early counts; (4) no test of canteen forecasts on unseen days. These are gaps of focus and application, not a claim that Poisson modelling of arrivals is new.

4 Data and Data Reference

Source and status. Field data collection at the canteen entrance is in progress. Until it is complete, the analysis uses a **synthetic prototype dataset**, `data/canteen_arrivals.csv`, generated by the project's `data_generator.py` with a fixed random seed (42). These values are not observed results; they are used to develop and check the methods, which apply unchanged to field counts.

Collection protocol (field data). Arrivals are counted at the canteen entrance, manually or from permitted video, in fixed 15-minute slots from 12:30 to 14:30 on m working days, following the observation style of Tang et al. [21].

How the prototype data were generated. 90 working days (one 18-week semester) $\times$ 8 slots. Expected arrivals follow a background of 14 per slot plus a Gaussian lunch rush centred at 13:15; each day gets a Gamma busyness factor (mean 1, coefficient of variation 5%) and a weekday effect (Friday 6% quieter); weeks 9 and 18 are exam weeks with demand reduced by 15%; counts are drawn from the resulting Poisson distributions; 0.5% of counts are blanked to exercise the cleaning step. The analysis below never uses these parameters: it re-estimates everything from the counts.

Cleaning. Missing, negative, fractional and duplicate counts are dropped rather than imputed, because imputing the mean would shrink the variance and bias the dispersion test. Of 720 records, 3 were missing, leaving 717 slot observations.

Table 2: Dataset variables

Column	Meaning
date	Calendar date of the working day
day	Weekday (derived from the date)
day_type	Regular or Exam (optional; Regular if absent)
time_interval	15-minute slot, HH:MM-HH:MM
arrivals	Number of students who entered in the slot
capacity	Capacity C per slot (optional)

Table 3: First five records

date	day	day_type	time_interval	arrivals	capacity
2026-01-05	Monday	Regular	12:30-12:45	32	30
2026-01-05	Monday	Regular	12:45-13:00	36	30
2026-01-05	Monday	Regular	13:00-13:15	28	30
2026-01-05	Monday	Regular	13:15-13:30	33	30
2026-01-05	Monday	Regular	13:30-13:45	34	30

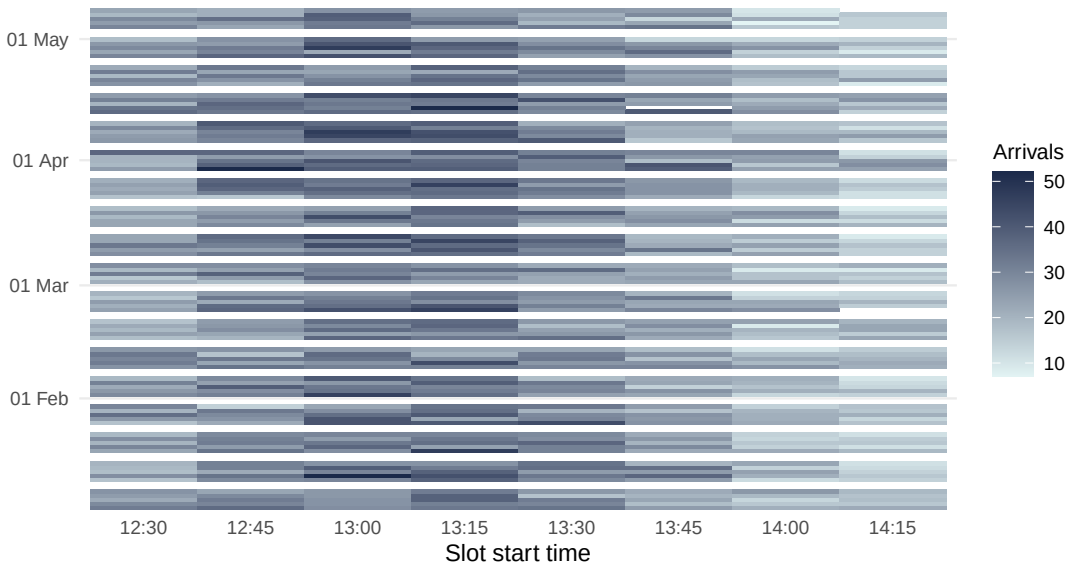


Figure 1: Every recorded count: days (rows) by slots (columns). The dark band is the lunch rush; paler rows are quieter days such as exam weeks.

5 Main Content: Analysis and Results

All results use capacity $C = 30$ per slot and significance level $\alpha = 0.05$.

5.1 Arrival rate per slot

Arrival rates rise from 24.5 at 12:30-12:45 to a peak of 34.4 at 13:15-13:30 and fall to 16.3 at 14:15-14:30 (Figure 2, Table 4).

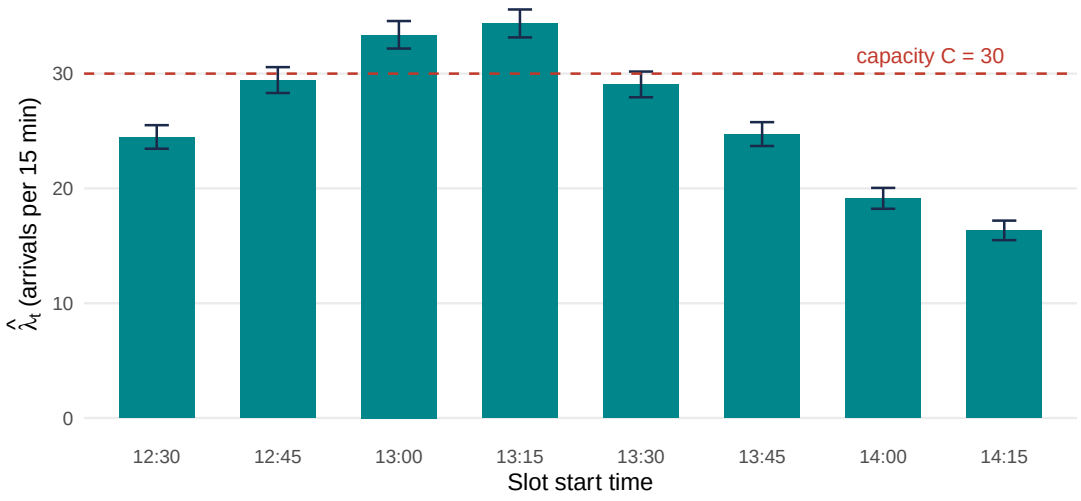


Figure 2: Estimated arrival rate per slot with exact 95% confidence intervals; the dashed line is the capacity.

5.2 Does the Poisson model fit?

The chi-square goodness-of-fit test rejects the Poisson model in 0 of 8 slots, against 0.4 rejections expected by chance alone (binomial check p = 1.000). Figure 3 compares the observed counts in the peak slot with the fitted Poisson distribution.

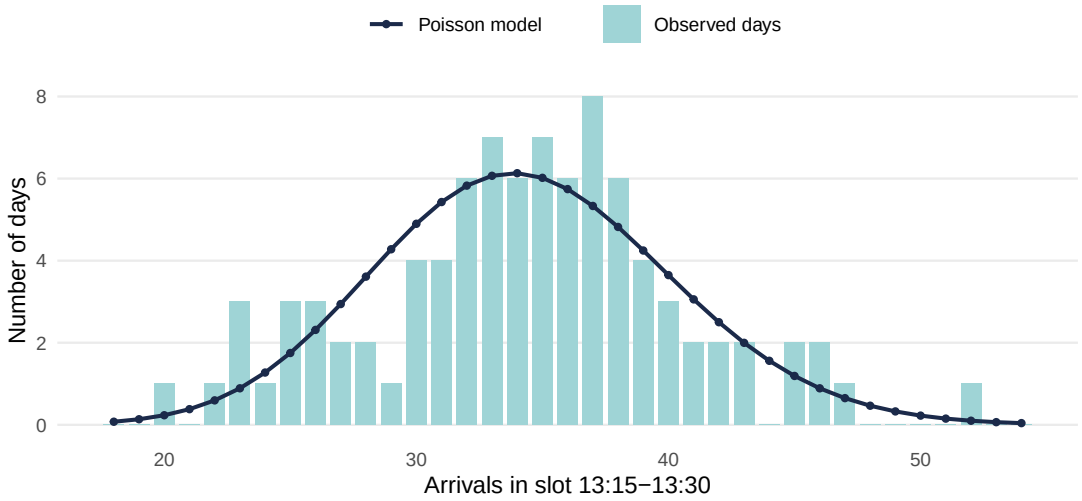


Figure 3: Observed distribution of daily counts in the peak slot against the fitted Poisson model.

Table 4: Per-slot estimates, Poisson checks and overload probability

Slot	$\hat{\lambda}_t$	95% CI	Var/Mean	Dispersion p	χ^2 GOF p	$P(X_t > C)$, %	Risk
12:30-12:45	24.47	[23.46, 25.51]	1.09	0.529	0.450	11.38	Low
12:45-13:00	29.42	[28.31, 30.56]	1.34	0.037	0.552	40.98	Medium
13:00-13:15	33.37	[32.18, 34.58]	1.26	0.100	0.309	68.23	High
13:15-13:30	34.36	[33.16, 35.59]	1.10	0.477	0.701	73.94	High
13:30-13:45	29.04	[27.94, 30.18]	1.03	0.817	0.218	38.26	Medium
13:45-14:00	24.72	[23.70, 25.77]	1.25	0.111	0.310	12.43	Low
14:00-14:15	19.12	[18.23, 20.05]	1.34	0.038	0.148	0.76	Low
14:15-14:30	16.33	[15.50, 17.20]	1.24	0.124	0.257	0.08	Low

5.3 Overload probability and risk

2 slot(s) are high risk, 2 medium and 4 low. The riskiest is 13:15-13:30 with $P(X > C) = 73.9\%$; across the confidence interval for λ it ranges from 66.9% to 80.1%. Keeping the overload probability below 20% in every slot would need a capacity of 39 per slot at the peak, against 30 at present.

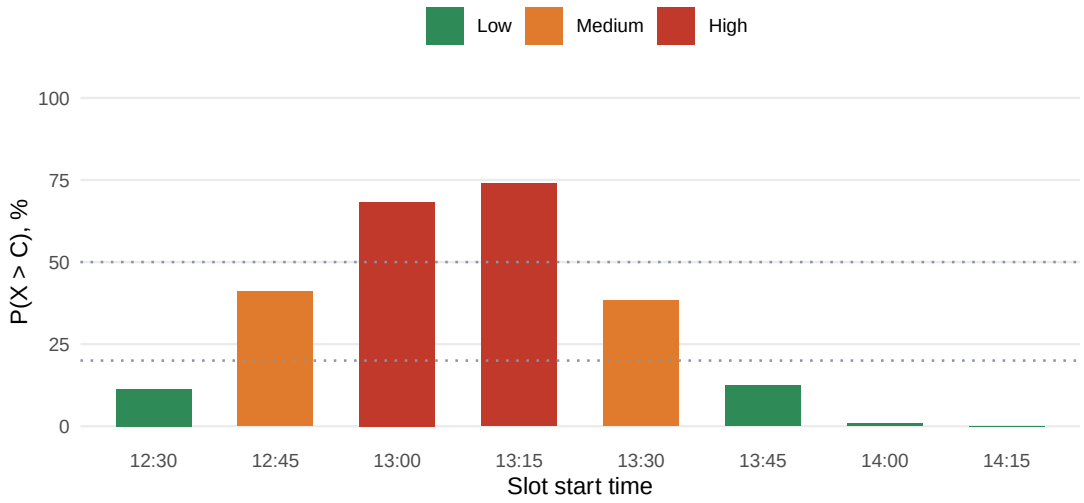


Figure 4: Overload probability per slot, coloured by risk level, with the 20% and 50% thresholds.

5.4 Least-crowded time

The quietest slot is 14:15-14:30 ($\hat{\lambda} = 16.33$). Conditional on the combined total, $S_1 \mid (S_1 + S_2) \sim \text{Binomial}(S_1 + S_2, m_1/(m_1 + m_2))$ under $H_0 : \lambda_1 = \lambda_2$ [17]; against the runner-up 14:00-14:15 ($\hat{\lambda} = 19.12$) the one-sided p-value is 5.2e-06, so it is significantly quieter.

5.5 Day-specific forecast by regression

To predict a particular day, log counts are regressed on dummy variables for slot, weekday and day type (reference: first slot, Monday, regular week): $\ln(x + 0.5) = \beta_0 + \alpha_t + \gamma_d + \delta_e + \varepsilon$. Fitted values are returned to the count scale with Duan's smearing factor 1.023 [7]. The model explains 57.9% of the variance (adjusted $R^2 = 0.572$; Table 5).

Table 5: Regression of $\ln(x + 0.5)$ on slot, weekday and day type

Term	Estimate	SE	t	p	$e^{\hat{\beta}}$
Intercept	3.2441	0.0282	114.90	< 0.001	25.638
Slot 12:45	0.1792	0.0324	5.53	< 0.001	1.196
Slot 13:00	0.3081	0.0324	9.51	< 0.001	1.361
Slot 13:15	0.3386	0.0324	10.45	< 0.001	1.403
Slot 13:30	0.1716	0.0324	5.30	< 0.001	1.187
Slot 13:45	0.0077	0.0325	0.24	0.812	1.008
Slot 14:00	-0.2570	0.0324	-7.93	< 0.001	0.773
Slot 14:15	-0.4100	0.0326	-12.59	< 0.001	0.664
Tuesday	-0.0179	0.0257	-0.69	0.487	0.982
Wednesday	0.0003	0.0257	0.01	0.991	1.000
Thursday	-0.0552	0.0257	-2.15	0.032	0.946
Friday	-0.0985	0.0257	-3.83	< 0.001	0.906
Exam week	-0.1198	0.0259	-4.62	< 0.001	0.887

On daily totals, the weekday effect gives one-way ANOVA F = 2.09 (p = 0.091) and Kruskal–Wallis H = 8.08 (p = 0.089). Exam days are quieter: 191.9 against 212.4 arrivals per lunch (Welch t = -4.37, p

< 0.001; Mann-Whitney p $= 0.005$).

5.6 Peak-time probability

For each kind of day, 20,000 lunch periods are simulated: draw the busyness factor W, then $X_t \sim \text{Poisson}(W\hat{\lambda}_t)$ for every slot. On a regular Monday the peak is most likely at 13:15-13:30 (42%), and the chance that at least one slot exceeds C is 97.9%, with 3.0 overloaded slots expected (Figures 5 and 6).

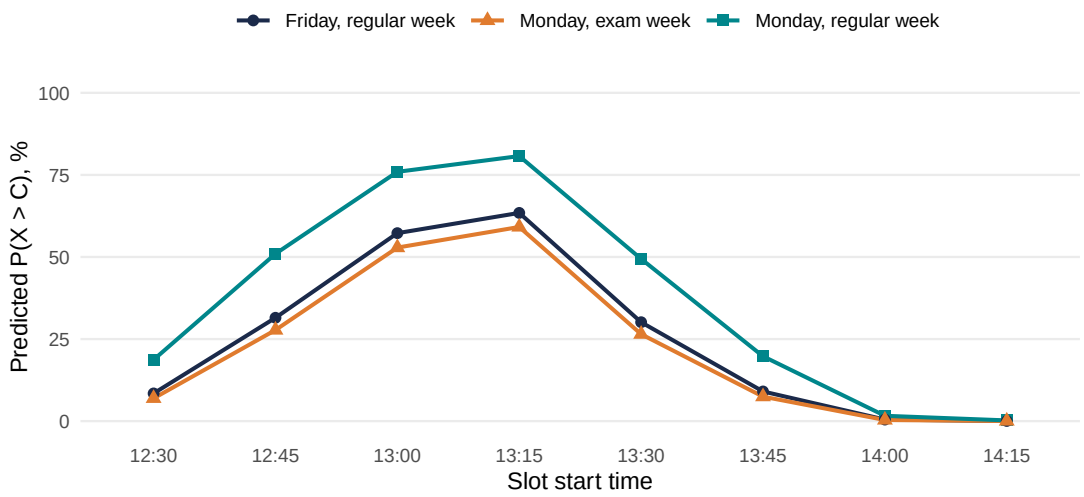


Figure 5: Predicted overload probability for three kinds of day (Poisson–Gamma model).

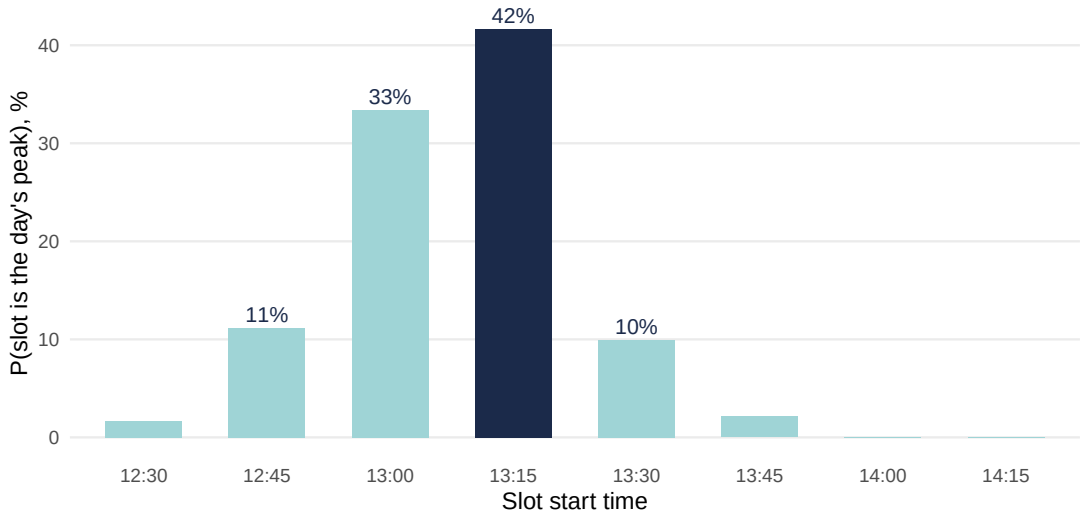


Figure 6: Probability that each slot is the busiest of the day on a regular Monday (20,000 simulated days).

5.7 Same-day Bayesian update

Suppose the first 2 slots of a regular Monday bring 30 and 36 students, about 15% more than forecast. The posterior mean of the busyness factor becomes 1.041, and the overload probability of each remaining slot rises (Table 6).

Table 6: Overload probability of the remaining slots before and after observing the first two slots

Remaining slot	Forecast λ_t	$P(X_t > C)$ before	$P(X_t > C)$ after
13:00-13:15	35.2	75.9%	82.6%
13:15-13:30	36.3	80.7%	86.6%
13:30-13:45	30.6	49.4%	57.8%
13:45-14:00	25.9	19.8%	25.3%
14:00-14:15	19.8	1.6%	2.4%
14:15-14:30	16.9	0.2%	0.3%

5.8 Validation on unseen days

Each day from day 31 onward is forecast from all earlier days only: 60 days and 477 slot forecasts. Table 7 summarises the hypothesis tests; Figures 7 and 8 show the calibration. Updating after the first 2 slots changes the Brier score of the later slots from 0.1240 to 0.1191.

Table 7: Hypothesis tests of the one-day-ahead forecasts (model vs baseline: MAE; hit rate vs chance)

Check	Test	Statistic	p	Result	Supports?
95% intervals cover actual counts	z, one proportion	z = -1.29	0.196	93.7%	Yes
Peak slot named correctly	z, one proportion	$z = 6.04$	< 0.001	40% vs 13%	Yes
Lower daily error than baseline	Paired t	$t = 0.56$	0.290	4.57 vs 4.62	No
More winning days than baseline	Sign test	35 wins / 60	0.123	35 of 60 days	No
PIT uniform	Kolmogorov-Smirnov	$D = 0.067$	0.027	not uniform	No
PIT histogram flat	χ^2 GOF	$\chi^2 = 20.38$	0.016	not flat	No

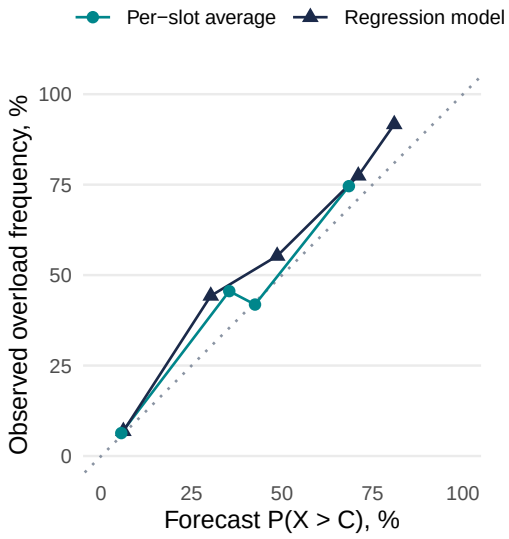


Figure 7: Reliability diagram: forecast overload probability against how often overload actually happened.

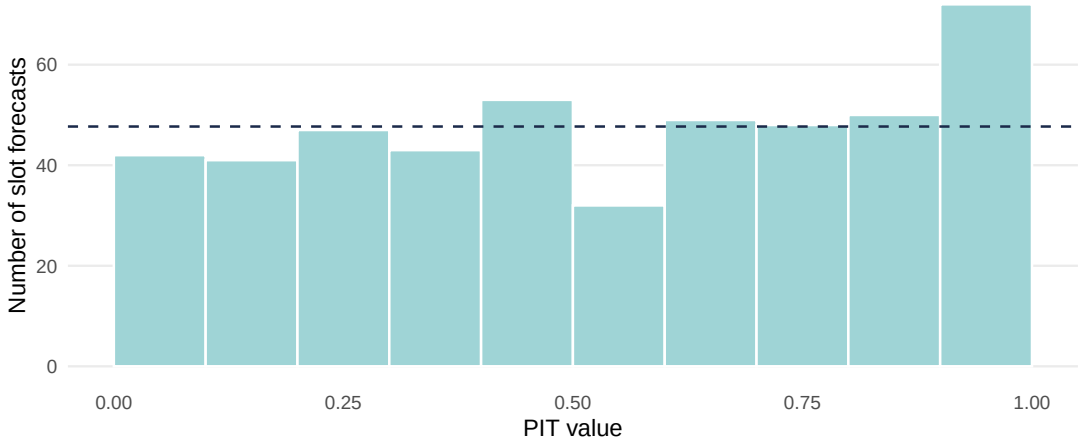


Figure 8: Randomised PIT values of the count forecasts; a flat histogram means a calibrated forecast distribution.

Carry-over and clustering. Spearman correlations of observed-to-predicted ratios between consecutive slots are significant for 2 of 7 pairs; the runs test on daily residual signs gives 42 runs against 45.8 expected (z = -0.81, p = 0.418).

6 Comparison

6.1 Poisson versus Poisson–Gamma

The Pearson test asks whether whole days vary more than the Poisson model allows once slot, weekday and exam effects are removed: $\chi^2 = 812.4$ on 704 df, p = 0.003, with estimated day-to-day variance $\hat{v}$ = 0.0057. Table 8 compares the two models' overload probabilities.

Table 8: Overload probability under the Poisson and Poisson–Gamma models

Slot	Poisson	Poisson–Gamma	Difference (points)
12:30-12:45	11.38%	12.84%	+1.47
12:45-13:00	40.98%	41.27%	+0.30
13:00-13:15	68.23%	66.48%	-1.76
13:15-13:30	73.94%	71.84%	-2.10
13:30-13:45	38.26%	38.75%	+0.49
13:45-14:00	12.43%	13.92%	+1.49
14:00-14:15	0.76%	1.09%	+0.33
14:15-14:30	0.08%	0.14%	+0.06

6.2 Forecast model versus the per-slot average

The per-slot average of past days (the "industry model" [1]) is the natural baseline. Figure 9 compares the two day by day, and Table 9 by kind of day.

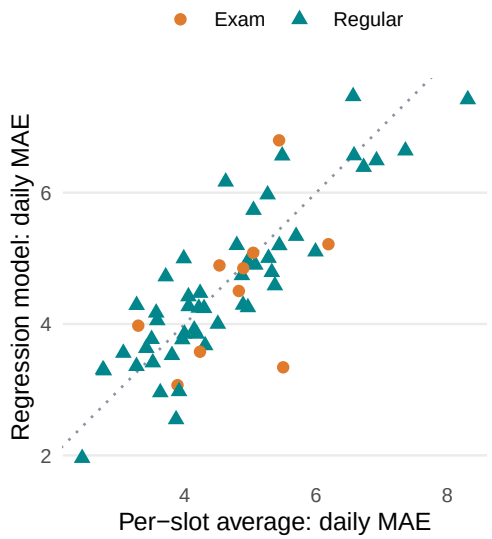


Figure 9: Daily mean absolute error of the regression model against the per-slot average; points below the diagonal are days where the model was more accurate.

Table 9: Mean absolute error by kind of day

Days	n	Model MAE	Baseline MAE
Exam days	10	4.53	4.78
Fridays	10	4.84	4.70
Other regular days	40	4.51	4.56

Table 10: Overall forecast accuracy on unseen days

	Brier score	Mean absolute error	95% interval coverage
Regression + Poisson–Gamma	0.1394	4.57	93.7%
Per-slot average + Poisson	0.1393	4.62	—

6.3 This work versus existing studies

Table 11: Comparison with existing studies

Study	Approach	Overload P	Tests Poisson	Peak time	Unseen days
Tang et al. [21]	Pedestrian flow in a canteen	No	No	No	No
Jiao et al. [11]	M/M/C simulation, one rate	No	No	No	No
Kim and Whitt [13]	Tests of Poisson arrivals	No	Yes	No	–
Weinberg et al. [24]	Bayesian rate forecasts	No	Partly	No	Yes
Shen and Huang [20]	Rate forecasts, intraday update	No	Partly	No	Yes
Shah et al. [19]	ML forecast of daily totals	No	No	No	Yes
This project	Per-slot Poisson + regression + Bayes	**Yes**	**Yes**	**Yes**	**Yes**

7 Discussion

The per-slot Poisson model fits the slots and turns arrival counts into a direct answer about overcrowding. Testing the assumption slot by slot, instead of building it in, is what makes the probabilities defensible, and the pooled Pearson test adds a check that per-slot tests alone can miss. The forecasting results are mixed and are reported as such: the forecast intervals are close to their nominal coverage, the busiest slot is named far more often than chance, the exam-week drop is captured, and the same-day update helps; but the gain in point accuracy over the per-slot average is small, in line with the observation that simple fixed-effect models are hard to beat when day effects are mild [10].

Limitations. (i) The data are synthetic; conclusions about the real canteen await field counts. (ii) The model estimates arrival risk, not queue length, which also depends on service rates and counters. (iii) The rate is assumed constant within a slot and arrivals independent, although friends often arrive together. (iv) The regression scales a whole day up or down and cannot move the peak to another slot on special days. (v) Capacity and risk thresholds are project decisions that must be justified for a specific canteen.

8 Conclusion

This project estimated the probability that canteen arrivals exceed capacity in each 15-minute slot using a Poisson model that was checked slot by slot, and extended it to predict a specific day's risk and peak time. On the prototype semester it identified a high-risk window around 13:00-13:15 and 13:15-13:30, a significantly quieter time at 14:15-14:30, the capacity needed at the peak (39 per slot for a 20% target), and a measurable benefit from updating the forecast with the day's first counts. Future work will replace the synthetic semester with field counts, add service times for an M/M/c queueing model, model group arrivals, and run the same-day update automatically from turnstile or camera counts.

References

[1] S. Aldor-Noiman, P. D. Feigin, and A. Mandelbaum, "Workload forecasting for a call center: Methodology and a case study," *The Annals of Applied Statistics*, vol. 3, no. 4, pp. 1403–1447, 2009, doi: 10.1214/09-AOAS255.

[2] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[3] L. Brown, N. Gans, A. Mandelbaum, A. Sakov, H. Shen, S. Zeltyn, and L. Zhao, "Statistical analysis of a telephone call center: A queueing-science perspective," *J. American Statistical Association*, vol. 100, no. 469, pp. 36–50, 2005, doi: 10.1198/016214504000001808.

[4] A. C. Cameron and P. K. Trivedi, "Regression-based tests for overdispersion in the Poisson model," *Journal of Econometrics*, vol. 46, no. 3, pp. 347–364, 1990, doi: 10.1016/0304-4076(90)90014-K.

[5] J. Chen and H. Wang, "Optimal management model of university canteen based on mixed queuing theory," *Journal of Huangshi Institute of Technology*, vol. 27, pp. 41–44, 2011.

[6] W. G. Cochran, "Some methods for strengthening the common χ^2 tests," *Biometrics*, vol. 10, no. 4, pp. 417–451, 1954, doi: 10.2307/3001616.

[7] N. Duan, "Smearing estimate: A nonparametric retransformation method," *J. American Statistical Association*, vol. 78, no. 383, pp. 605–610, 1983.

[8] F. Garwood, "Fiducial limits for the Poisson distribution," *Biometrika*, vol. 28, no. 3/4, pp. 437–442, 1936, doi: 10.1093/biomet/28.3-4.437.

[9] L. V. Green, P. J. Kolesar, and W. Whitt, "Coping with time-varying demand when setting staffing requirements for a service system," *Production and Operations Management*, vol. 16, no. 1, pp. 13–39, 2007, doi: 10.1111/j.1937-5956.2007.tb00164.x.

[10] R. Ibrahim, H. Ye, P. L'Ecuyer, and H. Shen, "Modeling and forecasting call center arrivals: A literature survey and a case study," *International Journal of Forecasting*, vol. 32, no. 3, pp. 865–874, 2016, doi: 10.1016/j.ijforecast.2015.11.012.

[11] S. Jiao, X. Wang, D. Song, and S. Du, "Simulation and optimization of university canteen service under the situation of epidemic prevention and control based on queuing theory," *ITM Web of Conferences*, vol. 47, 02052, 2022, doi: 10.1051/itmconf/20224702052.

[12] R. A. Johnson, *Miller & Freund's Probability and Statistics for Engineers*, 9th ed. Pearson Education, 2018.

[13] S.-H. Kim and W. Whitt, "Are call center and hospital arrivals well modeled by nonhomogeneous Poisson processes?" *Manufacturing & Service Operations Management*, vol. 16, no. 3, pp. 464–480, 2014, doi: 10.1287/msom.2014.0490.

[14] L. M. Leemis, "Nonparametric estimation of the cumulative intensity function for a nonhomogeneous Poisson process," *Management Science*, vol. 37, no. 7, pp. 886–900, 1991, doi: 10.1287/mnsc.37.7.886.

[15] D. S. Matteson, M. W. McLean, D. B. Woodard, and S. G. Henderson, "Forecasting emergency medical service call arrival rates," *The Annals of Applied Statistics*, vol. 5, no. 2B, pp. 1379–1406, 2011, doi: 10.1214/10-AOAS442.

[16] R. P. McNulty, "A new measure of forecast skill using the Brier score," Central Region Technical Attachment 88-26, National Weather Service, NOAA, 1988.

[17] J. Przyborowski and H. Wileński, "Homogeneity of results in testing samples from Poisson series," *Biometrika*, vol. 31, no. 3/4, pp. 313–323, 1940, doi: 10.1093/biomet/31.3-4.313.

[18] S. M. Ross, *Introduction to Probability Models*, 11th ed. Academic Press, 2014.

[19] H. Shah, N. Myller, and C. Islas Sedano, "Forecasting daily customer flow in restaurants: a multifactor machine learning approach," *Applied Computing and Intelligence*, vol. 5, no. 2, pp. 168–190, 2025, doi: 10.3934/aci.2025011.

[20] H. Shen and J. Z. Huang, "Forecasting time series of inhomogeneous Poisson processes with application to call center workforce management," *The Annals of Applied Statistics*, vol. 2, no. 2, pp. 601–623, 2008, doi: 10.1214/08-AOAS164.

[21] T.-Q. Tang, B.-T. Zhang, J. Zhang, and T. Wang, "Statistical analysis and modeling of pedestrian flow in university canteen during peak period," *Physica A*, vol. 521, pp. 29–40, 2019, doi: 10.1016/j.physa.2019.01.060.

[22] T.-Q. Tang, B.-T. Zhang, and C.-Z. Xie, "Modeling and simulation of pedestrian flow in university canteen," *Simulation Modelling Practice and Theory*, vol. 95, pp. 96–111, 2019, doi: 10.1016/j.simpat.2019.04.011.

[23] A. Variskhanov and T. Arreeras, "Early peak badges from Wi-Fi telemetry: A field feasibility study of lunchtime crowd management on a smart campus," *Urban Science*, vol. 10, no. 1, p. 29, 2026, doi: 10.3390/urbansci10010029.

[24] J. Weinberg, L. D. Brown, and J. R. Stroud, "Bayesian forecasting of an inhomogeneous Poisson process with applications to call center data," *J. American Statistical Association*, vol. 102, no. 480, pp. 1185–1199, 2007.

[25] N. Yuan, X. Wang, and X. Chen, "Analysis on the service capacity of university canteens under the epidemic prevention based on queue theory," *Transactions on Comparative Education*, vol. 5, no. 7, 2023, doi: 10.23977/trance.2023.050706.

Appendix: R Code

The complete R code that produces every number, table and figure in this document. To reproduce: open `canteenflow_report.Rmd` in RStudio (with `ggplot2` installed) and press **Knit**; the data file path is set in the `params` header.

```
# ---- Setup: packages, options, helpers ----------------------------------------
# Required packages: knitr, rmarkdown (come with RStudio) and ggplot2
#   (install.packages("ggplot2")).
library(knitr)
library(ggplot2)
opts_chunk$set(echo = FALSE, message = FALSE, warning = FALSE, fig.align = "center",
  fig.pos = "H",
               out.width = "85%", fig.width = 6.4, fig.height = 3.1, dev = "pdf")

C     <- params$capacity          # capacity per 15-minute slot
ALPHA <- params$alpha             # significance level
LOW   <- 0.20; HIGH <- 0.50       # risk thresholds for P(X > C)
INK <- "#1B2A4A"; TEAL <- "#00868B"; SOFT <- "#9FD4D6"; GREY <- "#8A94A3"
RISK_COL <- c(Low = "#2E8B57", Medium = "#E07B2E", High = "#C0392B")
DAYS <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday",
  "Sunday")

theme_report <- theme_minimal(base_size = 10) +
  theme(panel.grid.minor = element_blank(), panel.grid.major.x = element_blank(),
        legend.position = "top", legend.title = element_blank(), plot.title =
          element_blank())
pct  <- function(p, d = 1) sprintf(paste0("%.", d, "f%%"), 100 * p)
esc  <- function(x) gsub("%", "\\%", x, fixed = TRUE)    # for tables built with
  escape = FALSE
fmtp <- function(p) ifelse(p < 0.001, "< 0.001", sprintf("%.3f", p))
slot_lab <- function(s) s                                  # "12:30-12:45"
slot_start <- function(s) substr(s, 1, 5)                 # "12:30" (axis labels)
pv <- function(p) ifelse(p < 0.001, "p < 0.001", sprintf("p = %.3f", p))
tab <- function(x, caption, ...) kable(x, format = "latex", booktabs = TRUE, caption
  = caption, linesep = "", label = NA, row.names = FALSE, ...)
# ---- Data: read, clean and derive columns -------------------------------------
raw  <- read.csv(params$data, stringsAsFactors = FALSE)
n_in <- nrow(raw)
d <- raw[!is.na(raw$arrivals) & raw$arrivals >= 0 & raw$arrivals %% 1 == 0, ]
d <- d[!duplicated(d[c("date", "time_interval")]), ]
d$date <- as.Date(d$date)
d$day  <- c("Sunday", DAYS[1:6])[as.POSIXlt(d$date)$wday + 1]
if (!"day_type" %in% names(d)) d$day_type <- "Regular"
d$day_type[is.na(d$day_type) | d$day_type == ""] <- "Regular"
d <- d[order(d$date, d$time_interval), ]
slots  <- sort(unique(d$time_interval))
```

```r
K      <- length(slots)
n_days <- length(unique(d$date))
# ---- Per-slot Poisson estimation, exact (Garwood) CI and overload probability ----------
summ <- do.call(rbind, lapply(slots, function(s) {
  x <- d$arrivals[d$time_interval == s]; m <- length(x); S <- sum(x)
  data.frame(slot = s, m = m, total = S, lambda = S / m, var = var(x),
             ci_low = qchisq(ALPHA / 2, 2 * S) / (2 * m), ci_high = qchisq(1 - ALPHA / 2, 2 * S + 2) / (2 * m))
}))
summ$p_over      <- ppois(C, summ$lambda, lower.tail = FALSE)          # P(X > C) = 1 - P(X <= C)
summ$p_over_low  <- ppois(C, summ$ci_low, lower.tail = FALSE)
summ$p_over_high <- ppois(C, summ$ci_high, lower.tail = FALSE)
summ$risk <- ifelse(summ$p_over < LOW, "Low", ifelse(summ$p_over > HIGH, "High", "Medium"))

# ---- Goodness of fit: chi-square with categories merged until E >= 5 (Cochran), and dispersion test ----
gof_test <- function(x, min_exp = 5) {
  n <- length(x); lam <- mean(x)
  kmax <- max(max(x), qpois(1e-12, lam, lower.tail = FALSE)) + 1
  prob <- function(lo, hi) if (is.infinite(hi)) ppois(lo - 1, lam, lower.tail = FALSE) else
    ppois(hi, lam) - ppois(lo - 1, lam)
  singles <- c(lapply(0:(kmax - 1), function(k) c(k, k)), list(c(kmax, Inf)))
  groups <- list(); cur <- NULL
  for (s in singles) {
    cur <- if (is.null(cur)) s else c(cur[1], s[2])
    if (n * prob(cur[1], cur[2]) >= min_exp) { groups[[length(groups) + 1]] <- cur; cur <- NULL }
  }
  if (!is.null(cur)) groups[[length(groups)]][2] <- cur[2]      # leftover right tail joins the last group
  E <- sapply(groups, function(g) n * prob(g[1], g[2]))
  O <- sapply(groups, function(g) sum(x >= g[1] & x <= g[2]))
  chi2 <- sum((O - E)^2 / E); df <- length(groups) - 2
  list(lambda = lam, chi2 = chi2, df = df, p = pchisq(chi2, df, lower.tail = FALSE), k = length(groups))
}
disp_test <- function(x) {
  m <- length(x); D <- (m - 1) * var(x) / mean(x)
  c(index = var(x) / mean(x), D = D, p = min(1, 2 * min(pchisq(D, m - 1), pchisq(D, m - 1, lower.tail = FALSE))))
}
fits <- lapply(slots, function(s) gof_test(d$arrivals[d$time_interval == s]))
summ$gof_p  <- sapply(fits, `[[`, "p")
summ$disp_p <- sapply(slots, function(s) disp_test(d$arrivals[d$time_interval == s])["p"])
```

```r
rejections  <- sum(summ$gof_p <= ALPHA)
binom_p     <- pbinom(rejections - 1, K, ALPHA, lower.tail = FALSE)   # P(at least
↪  this many | all Poisson)

# ---- Least-crowded slot: exact conditional test against the runner-up
↪  ----------------------------
ranked <- summ[order(summ$lambda, summ$slot), ]
best <- ranked[1, ]; runner <- ranked[2, ]
quiet_p <- pbinom(best$total, best$total + runner$total, best$m / (best$m +
↪  runner$m))

# ---- Capacity planning: smallest C with P(X > C) below a target
↪  --------------------------------
required_capacity <- function(lam, target) {
  c <- max(1, qpois(1 - target, lam)); while (ppois(c, lam, lower.tail = FALSE) >=
↪  target) c <- c + 1; c }
summ$need20 <- sapply(summ$lambda, required_capacity, target = LOW)
peak <- summ[which.max(summ$lambda), ]; worst <- summ[which.max(summ$p_over), ]
# ---- Day-specific forecast: multiple linear regression on log counts
↪  ----------------------------
fit_model <- function(dd) {
  dd$slot <- factor(dd$time_interval, levels = sort(unique(dd$time_interval)))
  dd$dayf <- factor(dd$day, levels = DAYS[DAYS %in% dd$day])
  types   <- unique(dd$day_type)
  dd$typef <- factor(dd$day_type, levels = c(intersect("Regular", types),
↪  sort(setdiff(types, "Regular"))))
  terms <- c("slot", if (nlevels(dd$dayf) > 1) "dayf", if (nlevels(dd$typef) > 1)
↪  "typef")
  f <- lm(as.formula(paste("log(arrivals + 0.5) ~", paste(terms, collapse = " +
↪  "))), data = dd)
  smear <- mean(exp(resid(f)))                                    # Duan's smearing
↪  factor
  mu <- pmax(exp(fitted(f)) * smear - 0.5, 0.05)
  pearson <- sum((dd$arrivals - mu)^2 / mu)                       # Pearson
↪  chi-square, df = n - p
  v <- max((pearson - df.residual(f)) / sum(mu), 0)               # day-to-day
↪  variance Var(W)
  list(fit = f, smear = smear, mu = mu, v = v, pearson = pearson, dof =
   ↪  df.residual(f),
       slots = levels(dd$slot), days = levels(dd$dayf), types = levels(dd$typef),
        ↪  data = dd)
}
predict_day <- function(m, weekday, type = "Regular", at = m$slots) {
  nd <- data.frame(slot = factor(at, levels = m$slots), dayf = factor(weekday,
↪  levels = m$days),
                   typef = factor(type, levels = m$types))
  setNames(pmax(exp(predict(m$fit, nd)) * m$smear - 0.5, 0.05), at)
}
```

```r
p_over_mixed <- function(lam, C, v) if (v <= 1e-6) ppois(C, lam, lower.tail = FALSE)
↪  else
  pnbinom(C, size = 1 / v, mu = lam, lower.tail = FALSE)          # Poisson-Gamma =
  ↪  negative binomial

M <- fit_model(d)
v <- M$v
disp_p <- pchisq(M$pearson, M$dof, lower.tail = FALSE)
summ$p_mixed <- p_over_mixed(summ$lambda, C, v)
coefs <- summary(M$fit)$coefficients
r2 <- summary(M$fit)$r.squared; adj_r2 <- summary(M$fit)$adj.r.squared

# ---- Day effects on daily totals: ANOVA / Kruskal-Wallis (weekday), Welch t /
↪  Mann-Whitney (exam) ---
daily <- aggregate(arrivals ~ date + day + day_type, data = d, FUN = function(x)
↪  c(sum = sum(x), n = length(x)))
daily <- data.frame(daily[1:3], total = daily$arrivals[, "sum"], n =
↪  daily$arrivals[, "n"])
daily <- daily[daily$n == K, ]
reg_days <- daily[daily$day_type == "Regular", ]
aov_w <- oneway.test(total ~ day, data = reg_days, var.equal = TRUE)
kw_w  <- kruskal.test(total ~ day, data = reg_days)
has_exam <- any(daily$day_type != "Regular")
if (has_exam) {
  t_e <- t.test(daily$total[daily$day_type != "Regular"], reg_days$total, var.equal
↪  = FALSE)
  u_e <- wilcox.test(daily$total[daily$day_type != "Regular"], reg_days$total, exact
↪  = FALSE)
}

# ---- Peak-time probability by simulation, and P(at least one overload)
↪  --------------------------
peak_sim <- function(mu, v, n = 20000, seed = 2026) {
  set.seed(seed)
  W <- if (v > 1e-6) rgamma(n, shape = 1 / v, rate = 1 / v) else rep(1, n)
  X <- matrix(rpois(n * length(mu), outer(W, mu)), n, length(mu))
  pk <- max.col(X + matrix(runif(length(X), 0, 0.5), n), ties.method = "first")
  list(p_peak = setNames(tabulate(pk, length(mu)) / n, names(mu)),
       p_any = mean(rowSums(X > C) > 0), expected = mean(rowSums(X > C)))
}
cases <- data.frame(day = c("Monday", "Friday", "Monday"), type = c("Regular",
↪  "Regular", "Exam"))
if (!"Exam" %in% M$types) cases <- cases[1:2, ]
fc <- lapply(seq_len(nrow(cases)), function(i) {
  mu <- predict_day(M, cases$day[i], cases$type[i]); s <- peak_sim(mu, v)
  c(list(mu = mu, p = p_over_mixed(mu, C, v)), s) })
mon <- fc[[1]]
top2 <- names(sort(mon$p_peak, decreasing = TRUE))[1:2]
```

```r
# ---- Same-day Bayesian update: Gamma prior on the busyness factor W ----------------------------
mu_mon <- mon$mu; k_seen <- 2
obs_seen <- round(mu_mon[1:k_seen] * 1.15)                          # a morning 15% busier than forecast
a <- 1 / max(v, 1e-6)
post_shape <- a + sum(obs_seen); post_rate <- a + sum(mu_mon[1:k_seen])
rest <- mu_mon[-(1:k_seen)]
upd <- data.frame(slot = names(rest), mu = rest, before = p_over_mixed(rest, C, v),
                  after = pnbinom(C, size = post_shape, prob = post_rate / (post_rate + rest), lower.tail = FALSE))
# ---- Validation: forecast each day from earlier days only (expanding window, first 30 days training) ----
TRAIN <- 30; UPD <- 2
dates <- sort(unique(d$date))
set.seed(7)
rows <- list(); days_out <- list()
for (i in (TRAIN + 1):length(dates)) {
  train <- d[d$date < dates[i], ]; test <- d[d$date == dates[i], ]; test <- test[order(test$time_interval), ]
  m <- fit_model(train)
  if (!test$day[1] %in% m$days) next
  type <- if (test$day_type[1] %in% m$types) test$day_type[1] else "Regular"   # unseen exam type -> regular
  base_all <- tapply(train$arrivals, train$time_interval, mean)
  test <- test[test$time_interval %in% m$slots & test$time_interval %in% names(base_all), ]
  x <- test$arrivals; mu <- predict_day(m, test$day[1], type, test$time_interval)
  base <- as.numeric(base_all[test$time_interval]); vv <- m$v
  cdf <- function(q) if (vv <= 1e-6) ppois(q, mu) else pnbinom(q, size = 1 / vv, mu = mu)
  qf  <- function(p) if (vv <= 1e-6) qpois(p, mu) else qnbinom(p, size = 1 / vv, mu = mu)
  pit <- cdf(x - 1) + runif(length(x)) * (cdf(x) - cdf(x - 1))      # randomised PIT for counts
  p_upd <- rep(NA, length(x)); kk <- min(UPD, length(x) - 1)
  aa <- 1 / max(vv, 1e-6); sh <- aa + sum(x[1:kk]); rt <- aa + sum(mu[1:kk])
  p_upd[-(1:kk)] <- pnbinom(C, size = sh, prob = rt / (rt + mu[-(1:kk)]), lower.tail = FALSE)
  rows[[length(rows) + 1]] <- data.frame(
    date = dates[i], actual = x, mu = mu, base = base, p_model = p_over_mixed(mu, C, vv),
    p_base = ppois(C, base, lower.tail = FALSE), p_upd = p_upd, over = as.numeric(x > C),
    covered = as.numeric(x >= qf(0.025) & x <= qf(0.975)), pit = pit)
  days_out[[length(days_out) + 1]] <- data.frame(
    date = dates[i], day = test$day[1], day_type = test$day_type[1],
```

```
    mae_model = mean(abs(x - mu)), mae_base = mean(abs(x - base)),
    hit = x[which.max(mu)] == max(x), n_tied = sum(x == max(x)), n_slots =
    ↪ length(x))
}
S <- do.call(rbind, rows); Dd <- do.call(rbind, days_out)

brier <- function(p, o) mean((p - o)^2)
B_model <- brier(S$p_model, S$over); B_base <- brier(S$p_base, S$over)
U <- S[!is.na(S$p_upd), ]
B_upd <- brier(U$p_upd, U$over); B_noupd <- brier(U$p_model, U$over)
ztest <- function(succ, n, p0, alt = "two.sided") {
  z <- (succ / n - p0) / sqrt(p0 * (1 - p0) / n)
  c(phat = succ / n, z = z, p = if (alt == "greater") pnorm(z, lower.tail = FALSE)
  ↪ else 2 * pnorm(-abs(z))) }
cov_t  <- ztest(sum(S$covered), nrow(S), 0.95)
chance <- mean(Dd$n_tied / Dd$n_slots)
hit_t  <- ztest(sum(Dd$hit), nrow(Dd), chance, "greater")
pair_t <- t.test(Dd$mae_base, Dd$mae_model, paired = TRUE, alternative = "greater")
wins <- sum(Dd$mae_base > Dd$mae_model); losses <- sum(Dd$mae_base < Dd$mae_model)
sign_t <- binom.test(wins, wins + losses, 0.5, alternative = "greater")
ks_t   <- ks.test(S$pit, "punif")
pit_counts <- table(cut(S$pit, seq(0, 1, 0.1), include.lowest = TRUE))
chi_t  <- chisq.test(pit_counts)
cor_t  <- cor.test(S$mu, S$actual)
grp <- ifelse(Dd$day_type != "Regular", "Exam days", ifelse(Dd$day == "Friday",
↪ "Fridays", "Other regular days"))
mae_grp <- aggregate(cbind(mae_model, mae_base) ~ grp, data = cbind(Dd, grp = grp),
↪ FUN = mean)
mae_grp$n <- as.numeric(table(grp)[mae_grp$grp])

reliability <- function(p, o) {
  b <- findInterval(p, c(0.2, 0.4, 0.6, 0.8))
  do.call(rbind, lapply(sort(unique(b)), function(i)
    data.frame(bin = i, forecast = mean(p[b == i]), observed = mean(o[b == i]), n =
    ↪ sum(b == i))))
}

# ---- Dependence checks: consecutive-slot rank correlation, runs test on daily
↪ residuals ----------
d$ratio <- d$arrivals / M$mu
wide <- reshape(d[c("date", "time_interval", "ratio")], idvar = "date", timevar =
↪ "time_interval",
                direction = "wide")
wide <- wide[, c("date", paste0("ratio.", slots))]
cons <- do.call(rbind, lapply(1:(K - 1), function(j) {
  a1 <- wide[[j + 1]]; a2 <- wide[[j + 2]]; ok <- !is.na(a1) & !is.na(a2)
  ct <- suppressWarnings(cor.test(a1[ok], a2[ok], method = "spearman"))
  data.frame(pair = paste(slot_lab(slots[j]), "->", slot_lab(slots[j + 1])), rho =
  ↪ unname(ct$estimate),
```

```
            p = ct$p.value) }))
res_day <- aggregate(cbind(arrivals, mu = M$mu) ~ date, data = cbind(d, mu = M$mu),
↪  FUN = sum)
sgn <- res_day$arrivals > res_day$mu
n1 <- sum(sgn); n2 <- sum(!sgn); runs <- 1 + sum(sgn[-1] != sgn[-length(sgn)])
runs_mean <- 2 * n1 * n2 / (n1 + n2) + 1
runs_var  <- 2 * n1 * n2 * (2 * n1 * n2 - n1 - n2) / ((n1 + n2)^2 * (n1 + n2 - 1))
runs_z <- (runs - runs_mean) / sqrt(runs_var); runs_p <- 2 * pnorm(-abs(runs_z))
tab(data.frame(Column = c("date", "day", "day_type", "time_interval", "arrivals",
↪  "capacity"),
               Meaning = c("Calendar date of the working day", "Weekday (derived
               ↪  from the date)",
                           "Regular or Exam (optional; Regular if absent)",
                           ↪  "15-minute slot, HH:MM-HH:MM",
                           "Number of students who entered in the slot", "Capacity C
                           ↪  per slot (optional)")),
    "Dataset variables")
tab(head(raw[c("date", "day", "day_type", "time_interval", "arrivals", "capacity")],
↪  5), "First five records")
ggplot(d, aes(x = factor(slot_start(time_interval), levels = slot_start(slots)), y
↪  = date, fill = arrivals)) +
  geom_tile() + scale_fill_gradient(low = "#E3F4F4", high = INK, name = "Arrivals")
  ↪  +
  scale_y_date(date_labels = "%d %b", expand = c(0, 0)) + labs(x = "Slot start
  ↪  time", y = NULL) +
  theme_report + theme(legend.position = "right", legend.title = element_text(size =
↪  9))
ggplot(summ, aes(x = factor(slot_start(slot), levels = slot_start(slots)), y =
↪  lambda)) +
  geom_col(fill = TEAL, width = 0.62) +
  geom_errorbar(aes(ymin = ci_low, ymax = ci_high), width = 0.2, colour = INK) +
  geom_hline(yintercept = C, linetype = "dashed", colour = RISK_COL["High"]) +
  annotate("text", x = K, y = C + 1.6, label = paste("capacity C =", C), hjust = 1,
  ↪  size = 3,
           colour = RISK_COL["High"]) +
  labs(x = "Slot start time", y = expression(hat(lambda)[t] ~ "(arrivals per 15
  ↪  min)")) + theme_report
xp <- d$arrivals[d$time_interval == peak$slot]
kk <- seq(min(xp) - 2, max(xp) + 2)
ggplot() + geom_col(aes(x = kk, y = sapply(kk, function(k) sum(xp == k)), fill =
↪  "Observed days"), width = 0.8) +
  geom_line(aes(x = kk, y = length(xp) * dpois(kk, mean(xp)), colour = "Poisson
  ↪  model"), linewidth = 0.8) +
  geom_point(aes(x = kk, y = length(xp) * dpois(kk, mean(xp)), colour = "Poisson
  ↪  model"), size = 1) +
  scale_fill_manual(values = SOFT) + scale_colour_manual(values = INK) +
  labs(x = paste("Arrivals in slot", slot_lab(peak$slot)), y = "Number of days") +
  ↪  theme_report
```

```
tab(data.frame(Slot = slot_lab(summ$slot), lambda = sprintf("%.2f", summ$lambda),
               CI = sprintf("[%.2f, %.2f]", summ$ci_low, summ$ci_high),
               VarMean = sprintf("%.2f", summ$var / summ$lambda), Disp =
                ↪  fmtp(summ$disp_p),
               GOF = fmtp(summ$gof_p), P = sprintf("%.2f", 100 * summ$p_over), Risk
                ↪  = summ$risk),
    "\\label{tab:slots}Per-slot estimates, Poisson checks and overload probability",
    col.names = c("Slot", "$\\hat\\lambda_t$", "95\\% CI", "Var/Mean", "Dispersion
     ↪  $p$", "$\\chi^2$ GOF $p$",
                  "$P(X_t>C)$, \\%", "Risk"), escape = FALSE, align = "lccccccc")
ggplot(summ, aes(x = factor(slot_start(slot), levels = slot_start(slots)), y = 100
↪  * p_over, fill = risk)) +
  geom_col(width = 0.62) + geom_hline(yintercept = c(20, 50), linetype = "dotted",
   ↪  colour = GREY) +
  scale_fill_manual(values = RISK_COL, breaks = c("Low", "Medium", "High")) +
  labs(x = "Slot start time", y = "P(X > C), %") + ylim(0, 100) + theme_report
cf <- data.frame(Term = sub("^slot", "Slot ", sub("^dayf", "", sub("^typef", "",
↪  rownames(coefs)))),
                 Estimate = sprintf("%.4f", coefs[, 1]), SE = sprintf("%.4f",
                  ↪  coefs[, 2]),
                 t = sprintf("%.2f", coefs[, 3]), p = fmtp(coefs[, 4]), Multiplier =
                  ↪  sprintf("%.3f", exp(coefs[, 1])))
cf$Term <- sub("^Slot (\\d\\d:\\d\\d).*", "Slot \\1", cf$Term); cf$Term[1] <-
↪  "Intercept"
cf$Term[cf$Term == "Exam"] <- "Exam week"
tab(cf, "\\label{tab:coef}Regression of $\\ln(x + 0.5)$ on slot, weekday and day
↪  type", escape = FALSE,
    col.names = c("Term", "Estimate", "SE", "$t$", "$p$", "$e^{\\hat\\beta}$"),
     ↪  align = "lrrrrr")
fd <- do.call(rbind, lapply(seq_along(fc), function(i)
  data.frame(slot = slot_start(names(fc[[i]]$mu)), p = 100 * fc[[i]]$p,
             case = paste0(cases$day[i], ", ", tolower(cases$type[i]), " week"))))
ggplot(fd, aes(x = factor(slot, levels = slot_start(slots)), y = p, colour = case,
↪  group = case, shape = case)) +
  geom_line(linewidth = 0.8) + geom_point(size = 2) + scale_colour_manual(values =
   ↪  c(INK, RISK_COL[["Medium"]], TEAL)) +
  labs(x = "Slot start time", y = "Predicted P(X > C), %") + ylim(0, 100) +
   ↪  theme_report
pp <- data.frame(slot = factor(slot_start(names(mon$p_peak)), levels =
↪  slot_start(slots)), p = 100 * mon$p_peak)
ggplot(pp, aes(x = slot, y = p, fill = p == max(p))) + geom_col(width = 0.62,
↪  show.legend = FALSE) +
  geom_text(aes(label = ifelse(p >= 5, sprintf("%.0f%%", p), "")), vjust = -0.4,
   ↪  size = 3, colour = INK) +
  scale_fill_manual(values = c(`FALSE` = SOFT, `TRUE` = INK)) +
  labs(x = "Slot start time", y = "P(slot is the day's peak), %") + theme_report
tab(data.frame(Slot = slot_lab(upd$slot), Forecast = sprintf("%.1f", upd$mu), Before
↪  = esc(pct(upd$before)),
```

```
              After = esc(pct(upd$after))),
    "\\label{tab:update}Overload probability of the remaining slots before and after observing the first two slots",
    col.names = c("Remaining slot", "Forecast $\\lambda_t$", "$P(X_t>C)$ before", "$P(X_t>C)$ after"),
    escape = FALSE, align = "lccc")
yn <- function(ok) ifelse(ok, "Yes", "No")
tab(data.frame(
  Check = c("95\\% intervals cover actual counts", "Peak slot named correctly", "Lower daily error than baseline",
            "More winning days than baseline", "PIT uniform", "PIT histogram flat"),
  Test = c("$z$, one proportion", "$z$, one proportion", "Paired $t$", "Sign test", "Kolmogorov-Smirnov", "$\\chi^2$ GOF"),
  Statistic = c(sprintf("$z$ = %.2f", cov_t["z"]), sprintf("$z$ = %.2f", hit_t["z"]), sprintf("$t$ = %.2f", pair_t$statistic),
                sprintf("%d wins / %d", wins, wins + losses), sprintf("$D$ = %.3f", ks_t$statistic),
                sprintf("$\\chi^2$ = %.2f", chi_t$statistic)),
  p = fmtp(c(cov_t["p"], hit_t["p"], pair_t$p.value, sign_t$p.value, ks_t$p.value, chi_t$p.value)),
  Result = esc(c(pct(cov_t["phat"]), sprintf("%s vs %s", pct(hit_t["phat"], 0), pct(chance, 0)),
                 sprintf("%.2f vs %.2f", mean(Dd$mae_model), mean(Dd$mae_base)), sprintf("%d of %d days", wins, nrow(Dd)),
                 ifelse(ks_t$p.value < ALPHA, "not uniform", "uniform"), ifelse(chi_t$p.value < ALPHA, "not flat", "flat"))),
  OK = yn(c(cov_t["p"] >= ALPHA, hit_t["p"] < ALPHA, pair_t$p.value < ALPHA, sign_t$p.value < ALPHA,
            ks_t$p.value >= ALPHA, chi_t$p.value >= ALPHA)),
  "\\label{tab:valid}Hypothesis tests of the one-day-ahead forecasts (model vs baseline: MAE; hit rate vs chance)",
  col.names = c("Check", "Test", "Statistic", "$p$", "Result", "Supports?"), escape = FALSE, align = "lllrlc")
rel <- rbind(cbind(reliability(S$p_model, S$over), method = "Regression model"),
             cbind(reliability(S$p_base, S$over), method = "Per-slot average"))
ggplot(rel, aes(x = 100 * forecast, y = 100 * observed, colour = method, shape = method)) +
  geom_abline(linetype = "dotted", colour = GREY) + geom_line() + geom_point(size = 2) +
  scale_colour_manual(values = c(TEAL, INK)) + coord_equal(xlim = c(0, 100), ylim = c(0, 100)) +
  labs(x = "Forecast P(X > C), %", y = "Observed overload frequency, %") + theme_report
ggplot(data.frame(pit = S$pit), aes(x = pit)) +
  geom_histogram(breaks = seq(0, 1, 0.1), fill = SOFT, colour = "white") +
  geom_hline(yintercept = nrow(S) / 10, linetype = "dashed", colour = INK) +
  labs(x = "PIT value", y = "Number of slot forecasts") + theme_report
tab(data.frame(Slot = slot_lab(summ$slot), Poisson = pct(summ$p_over, 2), Mixed = pct(summ$p_mixed, 2),
```

```
              Diff = sprintf("%+.2f", 100 * (summ$p_mixed - summ$p_over))),
    "\\label{tab:mix}Overload probability under the Poisson and Poisson--Gamma
    ↪  models",
    col.names = c("Slot", "Poisson", "Poisson--Gamma", "Difference (points)"), align
    ↪  = "lccc")
ggplot(Dd, aes(x = mae_base, y = mae_model, colour = day_type, shape = day_type)) +
  geom_abline(linetype = "dotted", colour = GREY) + geom_point(size = 2) +
  scale_colour_manual(values = c(Exam = RISK_COL[["Medium"]], Regular = TEAL)) +
  ↪  coord_equal() +
  labs(x = "Per-slot average: daily MAE", y = "Regression model: daily MAE") +
  ↪  theme_report
tab(data.frame(Days = mae_grp$grp, n = mae_grp$n, Model = sprintf("%.2f",
↪  mae_grp$mae_model),
              Baseline = sprintf("%.2f", mae_grp$mae_base)),
    "\\label{tab:grp}Mean absolute error by kind of day", col.names = c("Days", "n",
    ↪  "Model MAE", "Baseline MAE"))
tab(data.frame(
  Study = c("Tang et al. [21]", "Jiao et al. [11]", "Kim and Whitt [13]", "Weinberg
  ↪  et al. [24]",
            "Shen and Huang [20]", "Shah et al. [19]", "\\textbf{This project}"),
  Approach = c("Pedestrian flow in a canteen", "M/M/C simulation, one rate", "Tests
  ↪  of Poisson arrivals",
               "Bayesian rate forecasts", "Rate forecasts, intraday update", "ML
               ↪  forecast of daily totals",
               "Per-slot Poisson + regression + Bayes"),
  Over = c("No", "No", "No", "No", "No", "No", "\\textbf{Yes}"),
  Tests = c("No", "No", "Yes", "Partly", "Partly", "No", "\\textbf{Yes}"),
  Peak = c("No", "No", "No", "No", "No", "No", "\\textbf{Yes}"),
  Valid = c("No", "No", "--", "Yes", "Yes", "Yes", "\\textbf{Yes}")),
  "Comparison with existing studies", escape = FALSE, align = "llcccc",
  col.names = c("Study", "Approach", "Overload $P$", "Tests Poisson", "Peak time",
  ↪  "Unseen days"))
```